



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS220: Database Systems

Class: BSAI – 2A

Lab 08: Data Modeling using ER Diagram

CLO-2: Formulate SQL queries to retrieve information from a relational database.

CLO 3: Build SQL (DDL &DML) queries to retrieve information from a relational database.

Date: 2 April 2026

Time: 10:00 - 01:00

Instructor: Sara Tariq Sheikh

Lab Engineer: Saood Sarwar



Lab 08: Data Modeling Using the Entity-Relationship (ER) Model

Introduction

An entity relationship diagram (ERD) shows the relationships of entity sets stored in a database. An entity in this context is a component of data. In other words, ER diagrams illustrate the logical structure of databases.

Objectives

After performing this lab, students should be able to:

1. Understand an entity relationship diagram (ERD)

Description

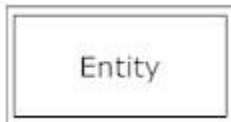
An entity relationship diagram is a means of visualizing how the information a system produces is related. There are five main components of an ERD:

- **Entities**, which are represented by rectangles. An entity is an object or concept about



which you want to store information.

- A weak entity is an entity that must be defined by a foreign key relationship with another entity, as it cannot be uniquely identified by its own attributes alone.



- **Actions**, which are represented by diamond shapes, show how two entities share

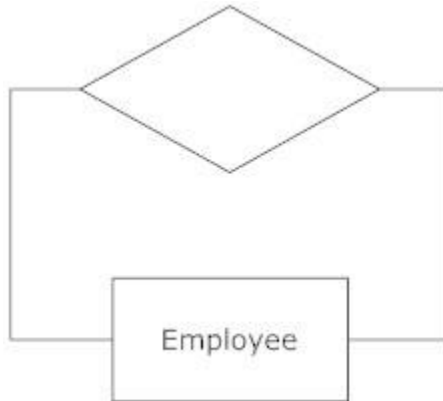


information in the database.



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

- In some cases, entities can be self-linked. For example, employees can supervise other employees.



- **Attributes**, which are represented by ovals. A key attribute is the unique, distinguishing characteristic of the entity. For example, an employee's social security number might be the employee's key attribute.



- A multivalued attribute can have more than one value. For example, an employee entity



can have multiple skill values.

- A derived attribute is based on another attribute. For example, an employee's monthly



salary is based on the employee's annual salary.

- **Connecting lines**, solid lines that connect attributes to show the relationships of entities in the diagram.
- **Cardinality** specifies how many instances of an entity relate to one instance of another entity. Ordinality is also closely linked to cardinality. While cardinality specifies the occurrences of a relationship, ordinality describes the relationship as either mandatory or optional. In other words, cardinality specifies the maximum number of relationships, and ordinality specifies the absolute minimum number of relationships.



Lab Tasks

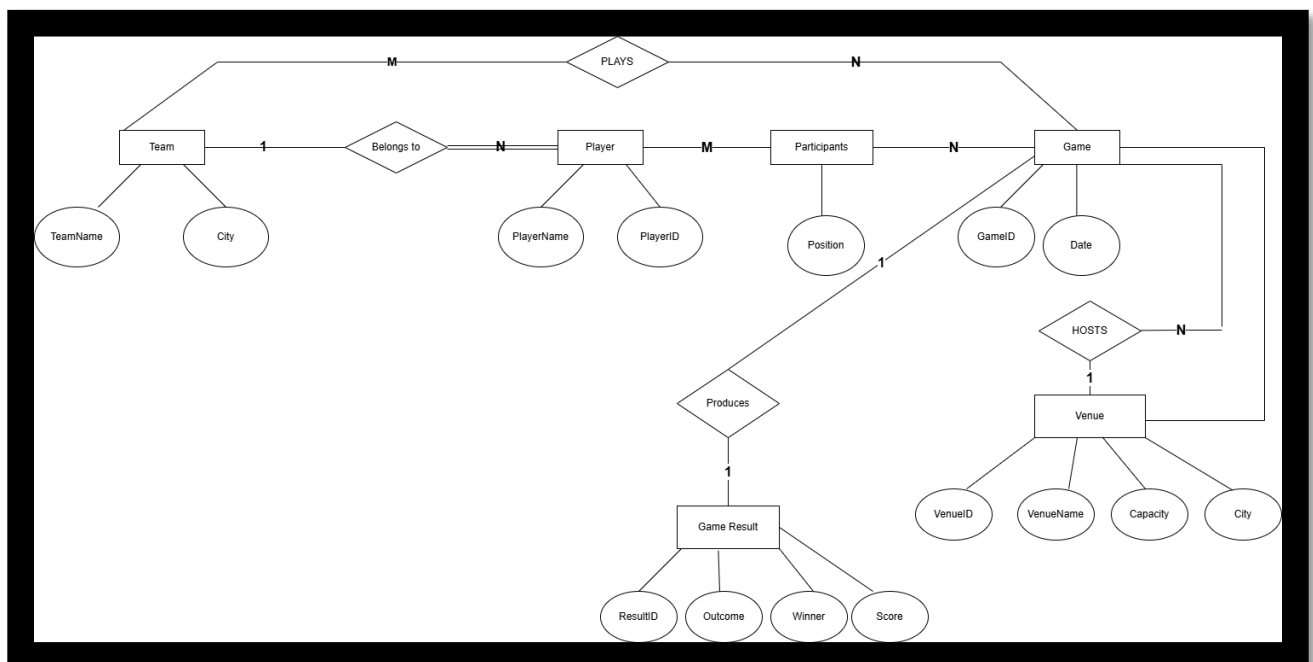
Given the scenario below, provide an **Entity-Relationship Diagram** that clearly shows all the entities and their relationships. Be sure that the structured constraints are shown. If you make any assumptions other than those described below, present them in writing. Be sure that these assumptions make sense. Also, make sure that the assumptions do not contradict any facts (implicit and explicit) given in the description.

Recommended: Try to use any available data modeling tool for the ER diagrams, such as IBM Rational Data Architect, ERwin, Microsoft Visio, DrawIO Extension on VSCode etc.).

Scenario 1

A database is being constructed to keep track of the teams and games of a sports league. A team has several players, not all of whom participate in each game. It is desired to keep track of the players participating in each game for each team, the positions they played in that game, and the result of the game. Choose your favorite sport (e.g., hockey, cricket, football, baseball, etc.).

ER MODEL:





Assumptions:

Primary Keys: Each entity (Team, Player, Game, Venue, and Game_Result) is uniquely identified by a specific ID (e.g., PlayerID, GameID) to maintain data integrity.

Team Membership: A player belongs to exactly one team during the season (1:N relationship). Total participation is required, meaning every player must be assigned to a team.

Game Structure: Each game involves two teams competing against each other. This is modeled as a Many-to-Many (M:N) relationship because a team plays many games, and a game involves multiple teams.

Player Participation: Not all players on a team roster participate in every game. The Position and specific match stats (Runs, Wickets) are stored on the Participants relationship because they vary for every match.

Venues: Each game is hosted at exactly one Venue (1:N). The Venue entity stores location-specific data like City and Capacity.

Results: There is a One-to-One (1:1) relationship between a Game and its Result. The Game_Result entity tracks the outcome, winner, and final score for each unique match.

Deliverables

1. Submit your lab tasks in one word file on LMS. The marking will be based on lab task submitted.